

2. Tecnologías para la programación en entorno cliente

2.1. Tecnologías web

Las tecnologías utilizadas en desarrollo web en entorno cliente son básicamente tres:

- **HTML.** Lenguaje de marcado utilizado para **estructurar** el contenido de la página.
- **CSS.** Lenguaje utilizado para crear los estilos y modificar el **diseño** (apariencia visual) de la página.



Apariencia visual

- **JavaScript.** Lenguaje de programación utilizado para añadir funcionalidad a la página. En ocasiones se utiliza algún otro lenguaje que puede ser compilado a JavaScript.

Junto a estas tecnologías básicas, nos encontraremos con **otras tecnologías** relacionadas con el desarrollo en entorno cliente:

- **WWW.** La *World Wide Web* es el mayor sistema software del mundo. Este sistema permite la comunicación entre **clientes** (navegadores web) y **servidores web** para intercambiar información a través del protocolo **HTTP**. Inicialmente, la información se mostraba a través del lenguaje HTML, denominado también **hipertexto**, aunque el concepto se ha ampliado al término **hipermedia**, ya que la información puede estar disponible en otros formatos de intercambio de datos enfocados a ser consumidos por una aplicación software, en lugar de ser mostrados directamente al usuario a través del navegador.

! IMPORTANTE

WWW no es sinónimo de Internet: Internet es una red global formada por la interconexión de multitud de redes y ordenadores a través de los protocolos IP y TCP/UDP; WWW es un sistema software (una «aplicación») que funciona sobre Internet, por lo que se sitúa en un nivel de jerarquía superior, que utiliza el protocolo HTTP a través de navegadores y servidores web.

- **HTTP.** HTTP es el protocolo utilizado por los navegadores y servidores web para comunicarse a través de la red. Es necesario estar familiarizado con él cuando se diseñan clientes que van a trabajar con **API de servidor**. Es un protocolo de tipo **half-duplex**, lo que significa que servidor y cliente no pueden comunicarse a la vez, sino que el servidor solo puede responder a peticiones previamente enviadas por el cliente.



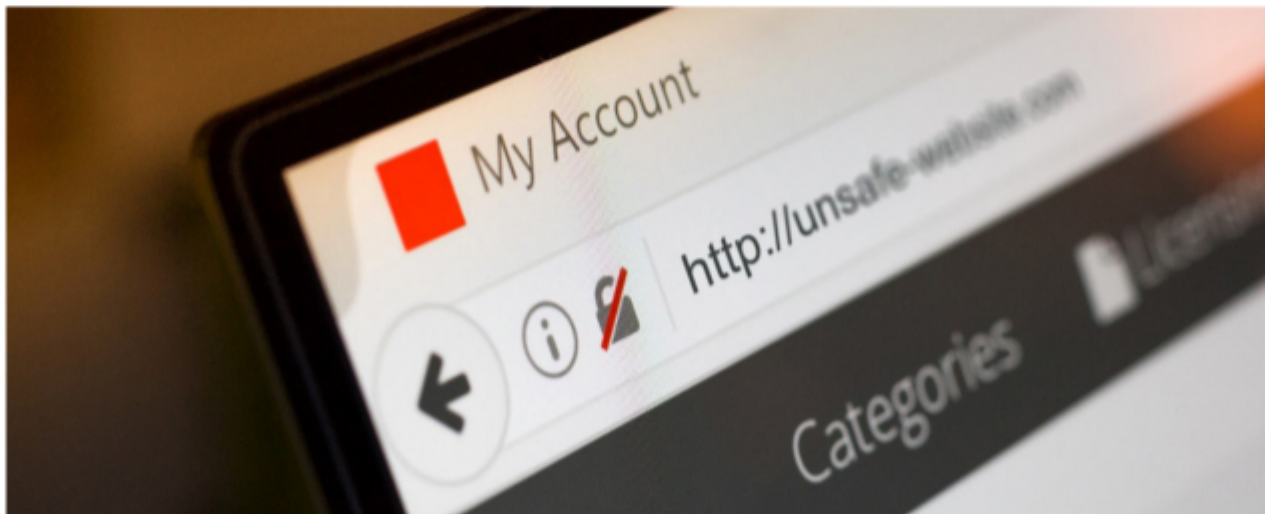
- **WebSocket.** Es un protocolo creado para que cliente y servidor puedan realizar comunicaciones **full duplex**: tanto cliente como servidor pueden enviar y recibir datos sin esperar a que haya una petición previa. Es muy útil en aplicaciones de tipo mensajería, ya que el servidor puede enviar los nuevos mensajes al cliente una vez los haya recibido, sin necesidad de que el cliente esté continuamente preguntando por actualizaciones.
- **URL.** Un *uniform resource locator*, o *identificador de recursos uniforme*, es una cadena de caracteres que **identifica un recurso web**. Este recurso web puede corresponder a una **página web** o a un **punto de acceso de una API web**. Una URL puede **responder a distintos tipos de peticiones** y puede **pro-vo-car** distintos tipos de **efectos**: puede generar una descarga de una página web, generar datos en un formato concreto, crear un recurso en una base de datos, activar un dispositivo, enviar una notificación, etcétera.
- **AJAX.** Asynchronous JavaScript and XML es un término que hace referencia a la capacidad de algunas páginas de enviar o recibir datos desde la lógica de servidor sin necesidad de solicitar la descarga completa de una página nueva: la página solo descarga los datos nuevos que necesita visualizar. No se trata de una tecnología adicional, sino más bien una técnica de programación que utiliza determinadas características de JavaScript.
- **CSS dinámico (SASS, LESS).** Son lenguajes que **extienden el lenguaje CSS** con características de lenguajes de programación, como variables, anidación o bucles.
- **Web API.** Una *web API* es un servicio de **lógica de servidor** que define unas URL a las que se pueden realizar **peticiones**. Normalmente, las peticiones de datos a API web no devuelven páginas web completas, sino **datos** en algún formato de intercambio, como **XML** o **JSON**, que deben ser **procesados por algún programa** (no van dirigidos a ser consumidos directamente por el usuario final).



IMPORTANTE

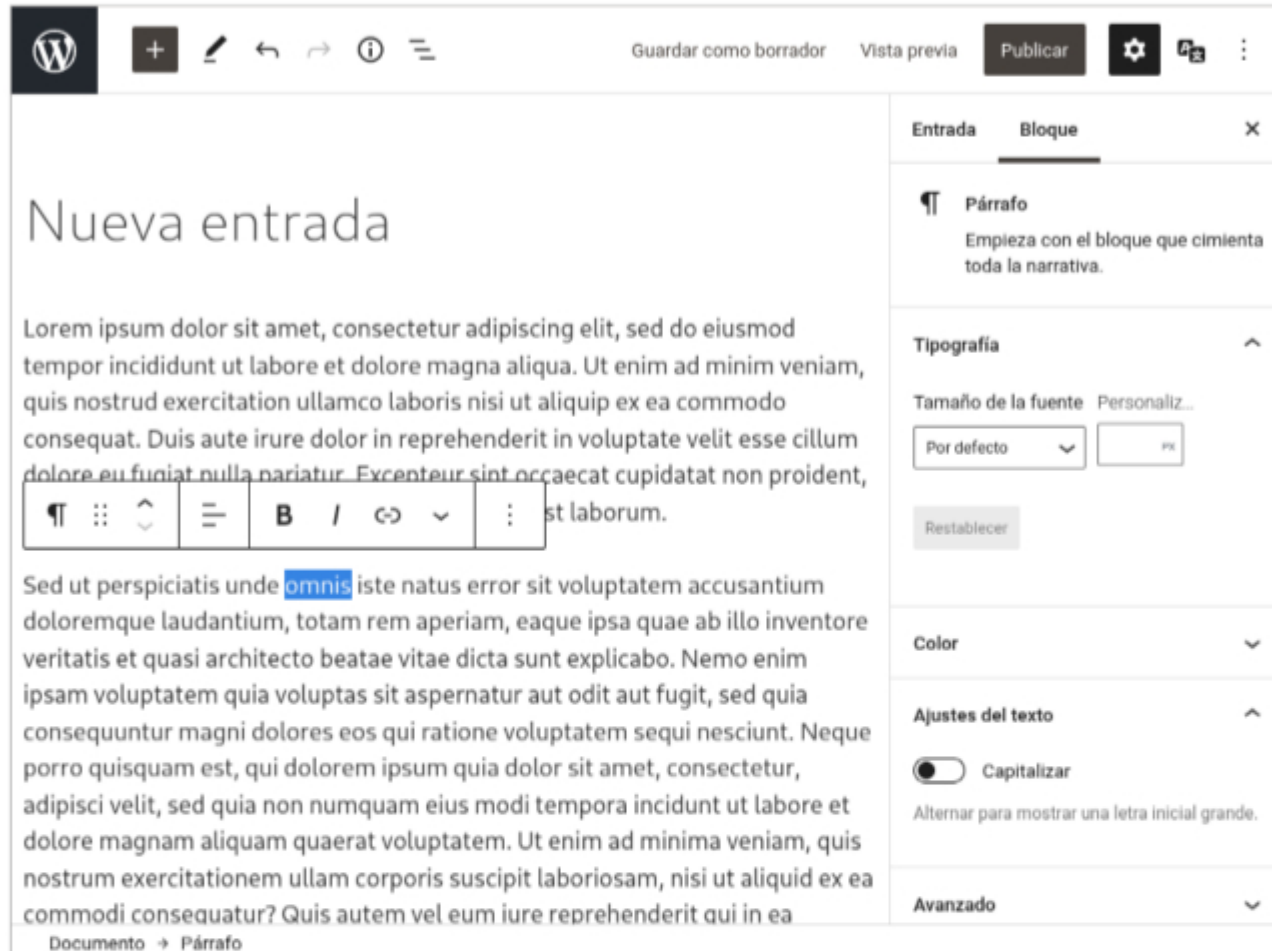
La ventaja de utilizar una API web en lugar de una aplicación de servidor «tradicional» es que la API puede proporcionar servicio a otros casos de uso, como pueden ser:

- Aplicaciones web de tipo SPA.
- Aplicaciones de escritorio.
- Aplicaciones móviles.
- Otros servicios web (como la publicación de mensajes de Facebook en un sitio web).





- **CMS.** Un CMS, *content management system* o **sistema de gestión de contenidos**, es una aplicación web que se utiliza para crear y administrar los contenidos de un sitio web. Este tipo de sistemas proporciona una serie de herramientas (editor web, interfaz para gestión de usuarios, etcétera) para que las personas encargadas de generar contenidos (artículos, posts, imágenes, archivos, etcétera) puedan hacerlo **sin necesidad de tener conocimientos técnicos**. **WordPress** es el gestor de contenidos más popular. Los gestores de contenidos normalmente utilizan **plantillas** para generar las páginas del sitio (se trataría de **páginas dinámicas**).



Plantilla

! IMPORTANTE

Dado que WordPress es una de las herramientas más utilizadas para crear sitios web, hay una gran demanda de empleo relacionado con esta tecnología. Las tareas del desarrollador de entorno cliente suelen estar relacionadas con la creación o la personalización de plantillas utilizadas por dicho gestor.

- **UI/UX.** Estos términos hacen referencia a la **interfaz de usuario** (*user interface*), o a la **experiencia de usuario** (*user experience*). Ambos términos están relacionados con el diseño de la parte de la aplicación que interacciona con el usuario, y hacen énfasis en su **apariencia visual, usabilidad y accesibilidad**. El término UI está más centrado en aspectos técnicos, mientras que el término UX tiene un sentido más amplio, ya que se enfoca en todos los aspectos que influyen en el usuario al utilizar la aplicación (por ejemplo, aspectos emocionales, como pueden ser el grado de satisfacción o frustración que experimenta en su manejo).
- **REST.** *Representational state transfer* es un estilo de arquitectura software que se utiliza para diseñar aplicaciones que sigan los estándares definidos para la comunicación a través de la WWW. Es un tipo de arquitectura muy utilizada para crear **API web**. La mayoría de los proveedores de servicios web (entre los que, por supuesto, están los grandes: Google, Twitter, Facebook, etcétera) ofrecen una API web de tipo REST para desarrollar aplicaciones que puedan operar con sus servicios.



- **SOAP.** *Simple object access protocol* es un protocolo de mensajería para intercambiar información entre clientes y servidores. El término suele hacer referencia también a un estilo de arquitectura software basado en este protocolo. En la mayoría de los artículos técnicos se define como un estilo **opuesto a REST**, en el sentido de que no aprovecha los estándares que gobiernan la WWW.
- **GraphQL.** Es un **lenguaje de consulta y manipulación de datos** para **API web**. En ocasiones, el término se utiliza también para definir API que utilizan dicho lenguaje.
- **XML/JSON.** XML y JSON son dos de los **lenguajes de intercambio de datos** más populares que existen. Se utilizan para **enviar datos que van a ser procesados por alguna aplicación**, en contraste con HTML, que se utiliza para enviar datos que van a mostrarse al usuario a través del navegador. Es habitual que las **API web** trabajen con datos en alguno de estos formatos.
- **Markdown.** Es un **lenguaje de marcado ligero** que se utiliza como **alternativa a HTML** por las personas encargadas de **escribir y editar artículos**. Se utiliza mucho en gestores de contenidos y en plataformas como GitHub, cuya lógica de servidor lo convierte a HTML para que pueda ser visualizado en navegadores.
- **YAML.** Es otro **lenguaje de marcado ligero** que se utiliza sobre todo en **ficheros de configuración** como **alternativa a XML**.

2.2. Lenguajes de programación en el entorno cliente

HTML es un lenguaje de marcado **declarativo**, por lo que solo describe **el contenido que debe aparecer en pantalla**: no tiene capacidad para interactuar con el usuario, aparte de cambiar la apariencia, enviar formularios o acceder a otras páginas a través de enlaces. Por ello, desde la aparición de los primeros navegadores, se desarrollaron tecnologías para dotar a las páginas web de soporte para lenguajes de programación que permitieran ofrecer las funcionalidades que faltaban. Como suele ser habitual, al principio cada compañía apostó por su tecnología propia. Algunas de dichas tecnologías son:

- **JavaScript.** Es la tecnología original que en un principio fue desarrollada por el navegador Netscape. Debido a su popularidad, se estandarizó en **ECMAScript**.
- **Jscript.** Es una versión de JavaScript que se creó a partir del original para el navegador Internet Explorer a principios de los años 2000.
- **ActionScript.** Es un lenguaje derivado de JavaScript que se utilizó principalmente en Adobe Flash, ahora abandonado.
- **Java.** Java es un lenguaje de programación de alto nivel muy utilizado como lógica de servidor. Durante años fue utilizado para la tecnología de cliente Java Applets, ahora abandonada.
- **TypeScript.** Es una expansión de JavaScript creada por Microsoft con el objetivo de añadir, entre otras cosas, **tipos de datos estáticos** (JavaScript es un lenguaje de tipos dinámicos). Para su ejecución en navegadores, es necesario **compilarlo** (transpilarlo) a JavaScript.
- **CoffeeScript.** Es un lenguaje que versiona JavaScript con el objetivo de hacerlo **más conciso y breve**. Debe **compilarse a JavaScript** para que pueda ser ejecutado en navegadores.
- **WebAssembly.** De reciente aparición (2017), no es un lenguaje de programación propiamente dicho, sino un **formato de código binario** para ejecución de programas en el navegador. Su objetivo es poder ejecutar programas binarios creados a partir de la compilación de lenguajes de alto nivel (C, C++, Rust, etcétera). WebAssembly ofrece mayor velocidad y potencia que JavaScript, por lo que su uso está recomendado para **juegos** o programas que necesiten gran capacidad de procesamiento.

En la actualidad, JavaScript es el lenguaje de cliente más utilizado para la programación en entorno cliente. TypeScript se utiliza sobre todo en proyectos complejos y en algunos *frameworks* de desarrollo, como **Angular**.

! IMPORTANTE

JavaScript es un lenguaje compilado, no interpretado: el código fuente del programa se compila (convierte a código máquina) justo antes de ejecutarse. El proceso se denomina compilación en tiempo de ejecución, just-in-time compilation.